

Computer Vision Project

Project Name: Perceptual Similarity with Triplet Learning

Team: Han Huawei, 2272375, han.2272375@studenti.uniroma1.it

Goal

Perceptual similarity models rely on ViTs, but full self-attention creates high computational cost.

Efficient Attention variants are proposed to reduce quadratic computational complexity.

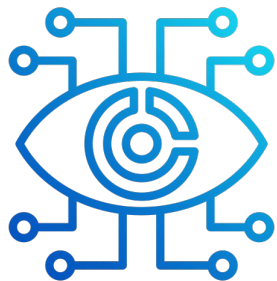
Can efficient-attention variants boost or maintain perceptual-similarity accuracy while keeping compute low?

Experiment Setup

Setting	Plan	Effective after budget ablation analysis
Dataset	20k triplets	4k triplets
Backbone	DeiT-Tiny / Small / Base	DeiT-Small
Attention	Standard / MoH / Pyra / Meta	Standard / MoH / Pyra / Meta
Layer replacement	All / first / last / every-other	every-other

Finally, hyper parameters are selected based on accuracy of some initial experiments. In which learning rate is fixed to $1e-5$, router learning rate is fixed to $3e-5$, min learning rate is fixed to $1e-7$, with weight decay at 0.05 and warmup ratio at 0.05.

Method Overview: DreamSim Pipeline



Computer Vision

pixels, patches.

low-level colors
and textures.



Which is more similar?

Perceptual Similarity



Human Vision

from visual
signals, to colors,
contrast, edges,
shapes, objects,
finally meaning.

Method Overview: DreamSim Pipeline

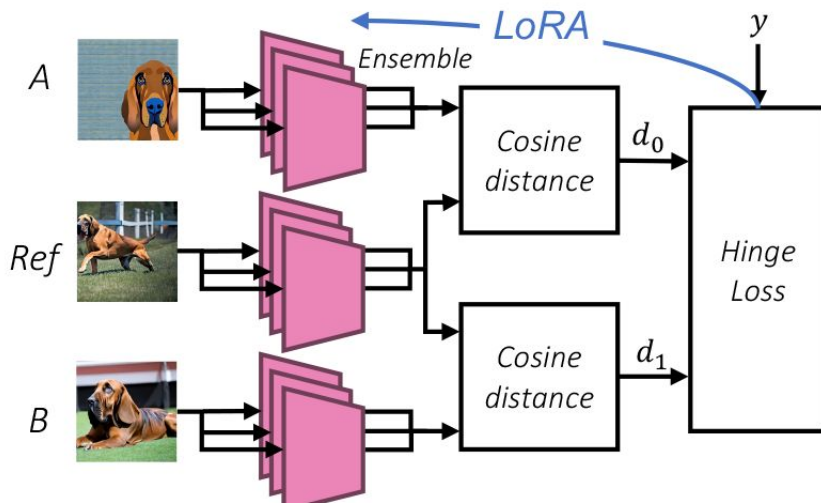
DreamSim pipeline

Each input triplet (Ref, A, B) is encoded using a shared ViT encoder.

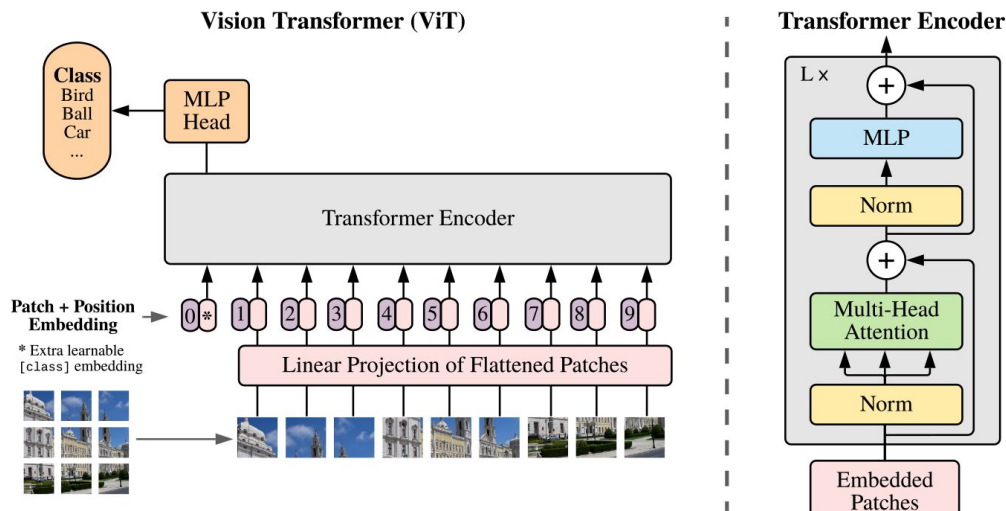
The cosine similarities between the reference image and the two candidates are computed: $S_a = \cos(\text{Ref}, A)$ and $S_b = \cos(\text{Ref}, B)$.

The ViT encoder is fine-tuned using LoRA on perceptual similarity data.

The fine-tuned model is then used to estimate perceptual similarity between the reference and candidate images.



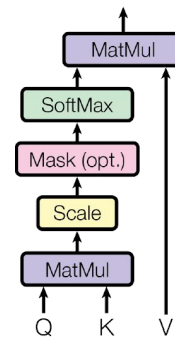
Method Overview: Efficient Attention Variants



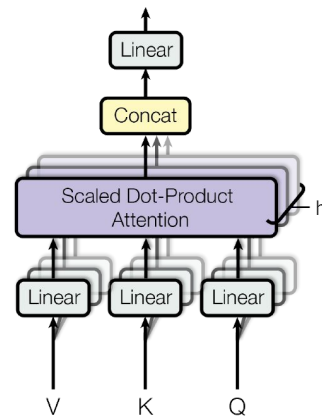
In ViT-based networks, images are divided into a large number of patches, this results in significant computational overhead.

Attention mechanism presents a computational problem due to its quadratic complexity relative to the input tokens.

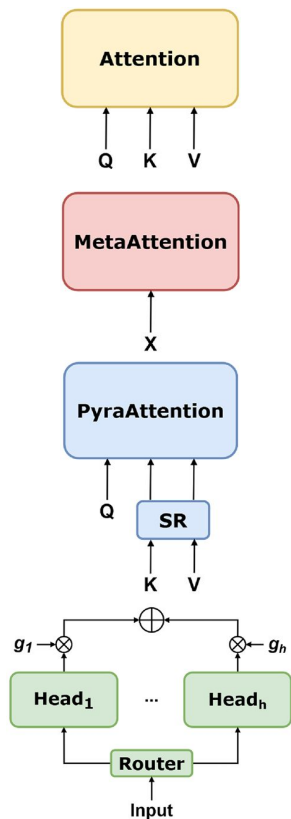
Scaled Dot-Product Attention



Multi-Head Attention



Method Overview: Efficient Attention Variants



$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^\top}{\sqrt{d_h}} \right) V$$

$$\text{MetaAttention}(x) = \text{Pooling}_k(x)$$

$$\text{PyraAttention}(Q, K, V) = \text{Softmax} \left(\frac{QSR(K)^\top}{\sqrt{d_h}} \right) SR(V)$$

$$SR(x) = \text{Norm}(\text{Reshape}(x, R)W)$$

$$\text{MoHHead}(Q, K, V) = \text{Concat}(H^1, H^2, \dots, H^h) W_O$$

$$H^i = \text{Attention}(Q^i, K^i, V^i)$$

Method Overview: Efficient Attention Variants

- Standard Attention: Full attention $\rightarrow$ quadratic complexity with respect to the number of tokens.
- Meta: Lightweight MLP router $\rightarrow$ routes heads and performs efficient token mixing.
- Pyra: Spatially reduced attention $\rightarrow$ progressively reduces input resolution; uses a pyramidal prior.
- MoH: Head routing $\rightarrow$ selectively activates relevant heads, with optional top-k selection.

Metrics

2AFC accuracy

Two-Alternative Forced Choice accuracy

Compare model choice with human choice

Measures how often the model agrees with the human preference

Inference speed

Inference latency = how long the model takes to process one input

Inference throughput = how many inputs the model can process per second

GPU memory usage

Peak GPU memory allocated by PyTorch during the inference run

Results

Ablation Study(MoH only)

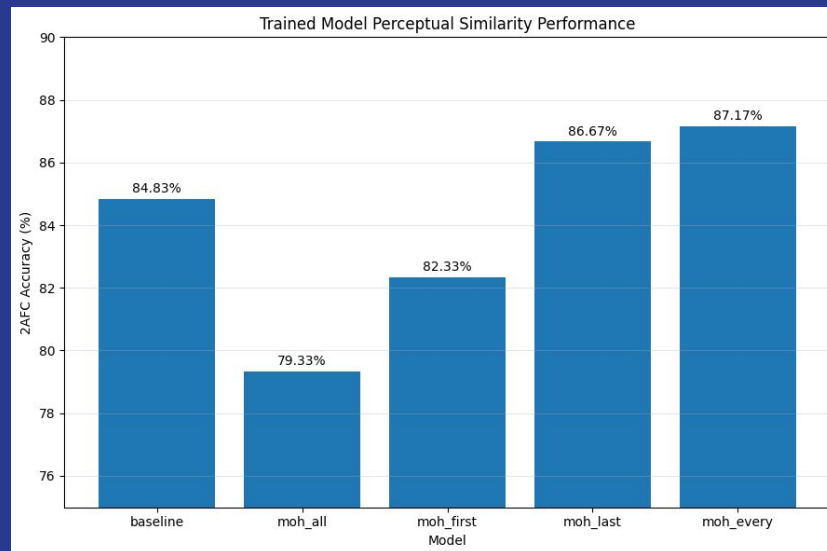
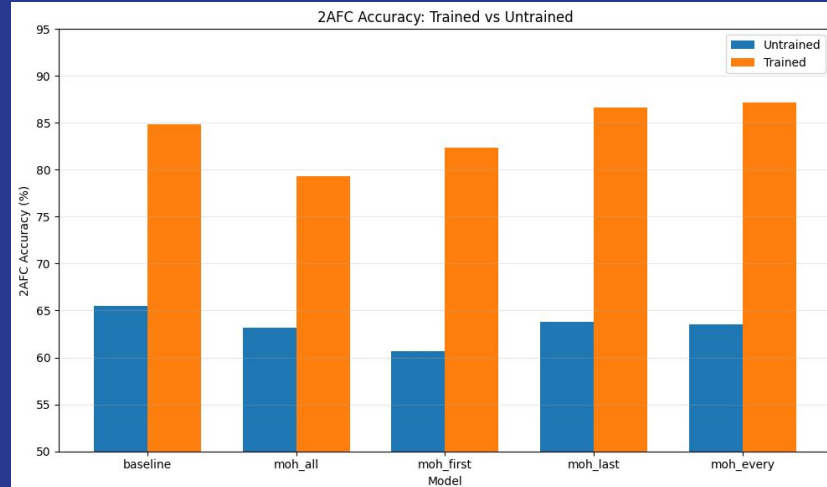
Accuracy

MoH-every improves from 84.83% to 87.17% (+2.758 %)

MoH-last improves from 84.83% to 86.67% (+2.169 %)

MoH-first improves from 84.83% to 82.33% (-2.947 %)

MoH-all improves from 84.83% to 79.33% (-6.484 %)



Inference speed

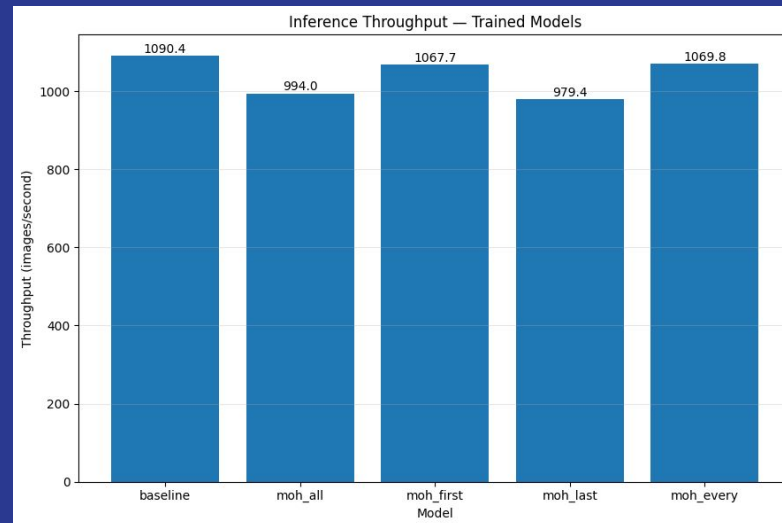
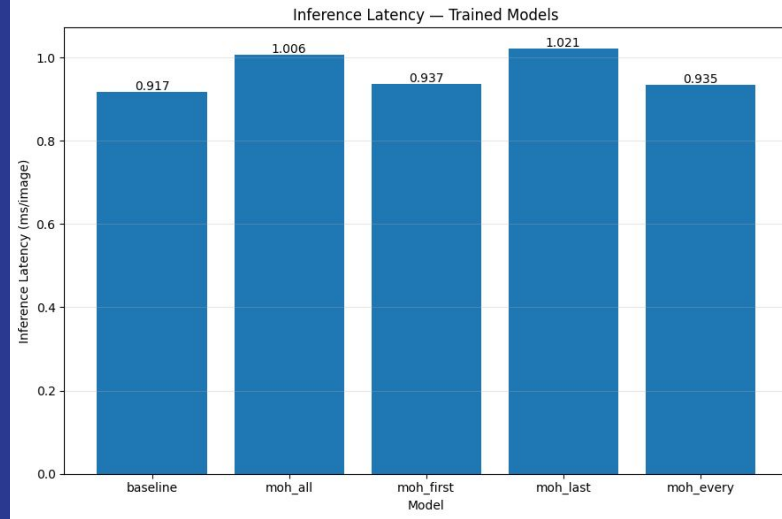
Inference Latency

MoH-every increases from 0.917 to 0.935 (+1.963 %)

MoH-first increases from 0.917 to 0.937 (+2.181 %)

MoH-all increases from 0.917 to 1.006 (+9.706 %)

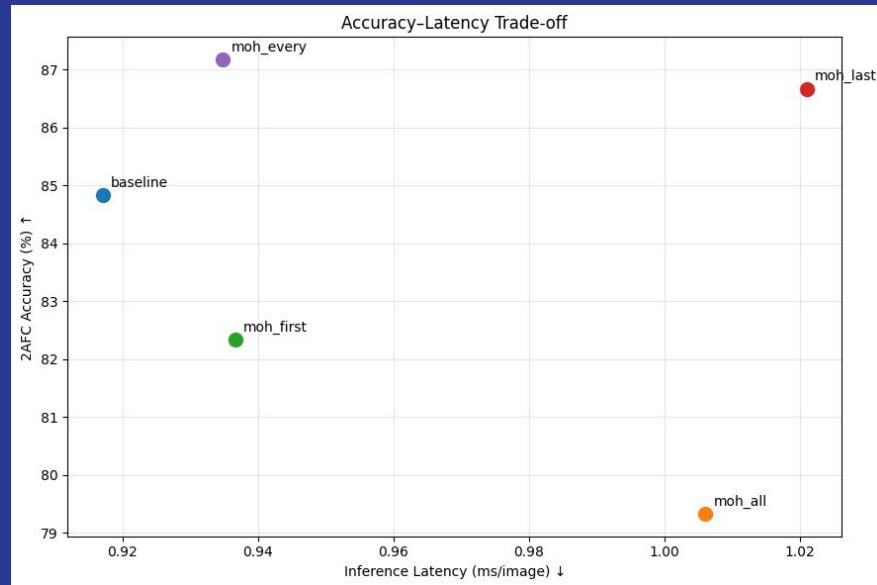
MoH-last increases from 0.917 to 1.021 (+11.341 %)



Trade-offs

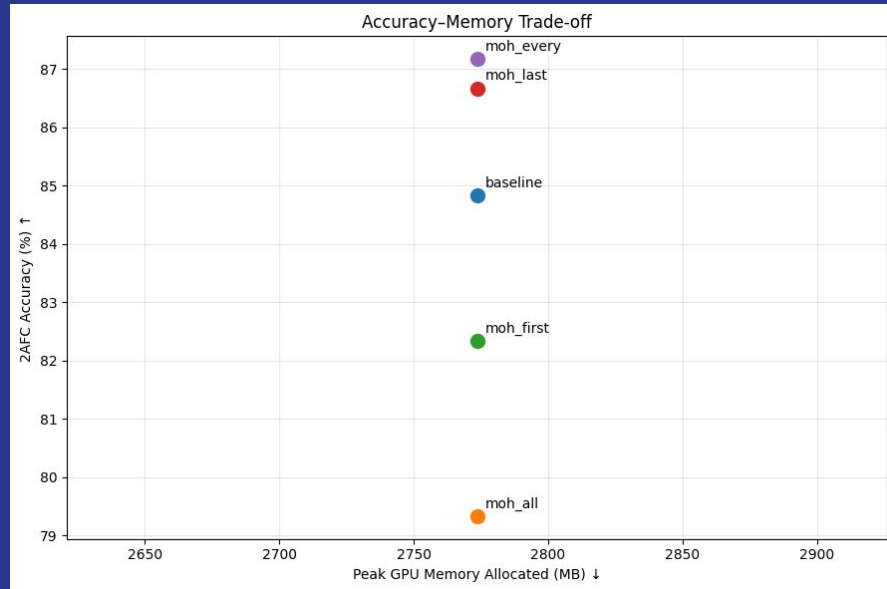
MoH-every improves 2AFC accuracy from 84.83% to 87.17% (+2.33 %), while increasing inference latency by only 1.93% and reducing throughput by only 1.90%.

So later in training other models, we used only the layer replacement strategy of **every** other layer.



Memory Usage

Peak GPU memory was not used for comparative analysis because the measured peak allocation was identical across all evaluated variants.



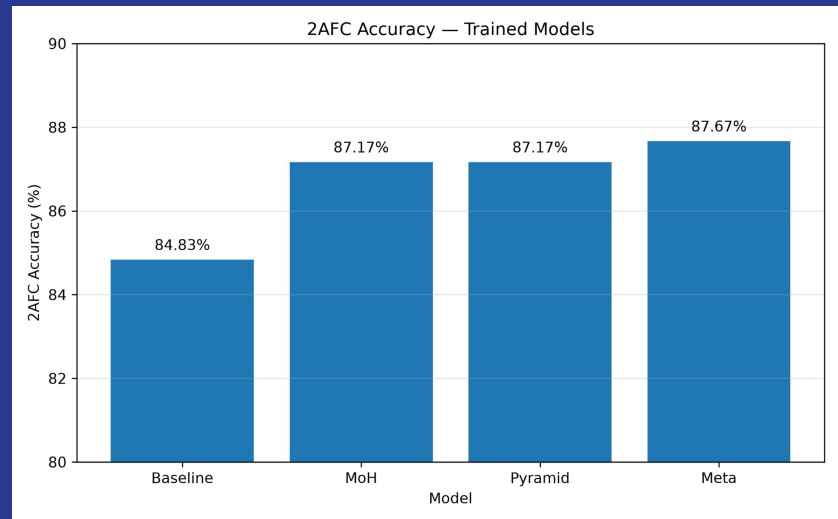
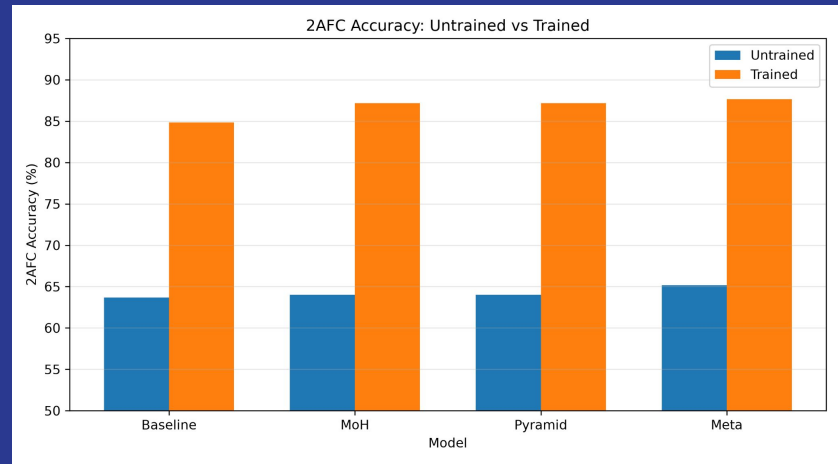
Full Comparison(MoH/Pyra/Meta)

Accuracy

Meta-every improves from 84.83% to 87.67% (+3.348 %)

MoH-every improves from 84.83% to 87.17% (+2.758 %)

Pyramid-every improves from 84.83% to 87.17% (+2.758 %)



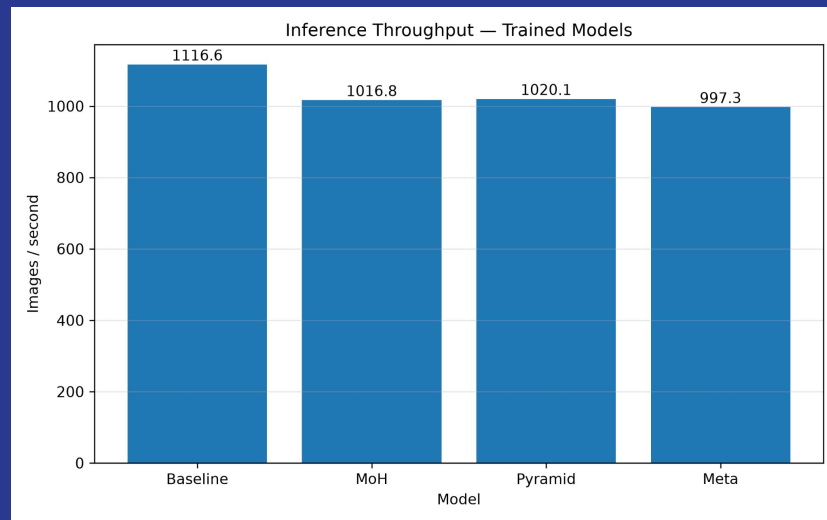
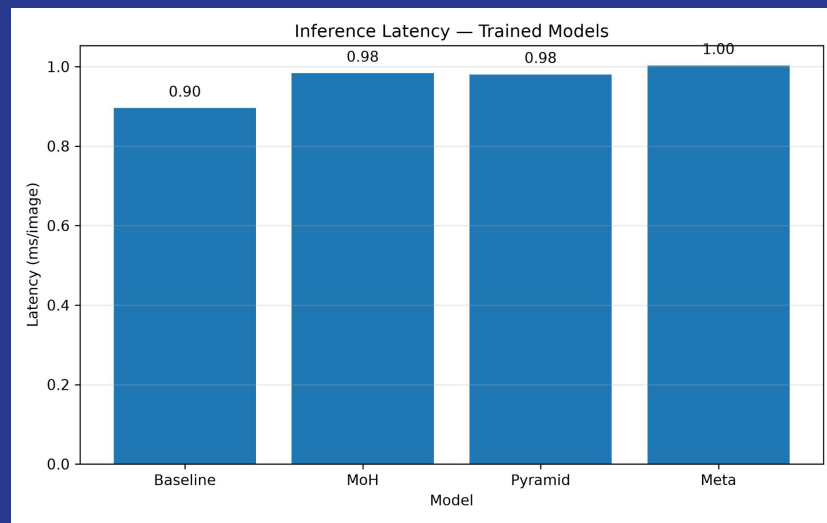
Inference speed

Inference Latency

MoH-every increases from 0.90 to 0.98 (+8.889 %)

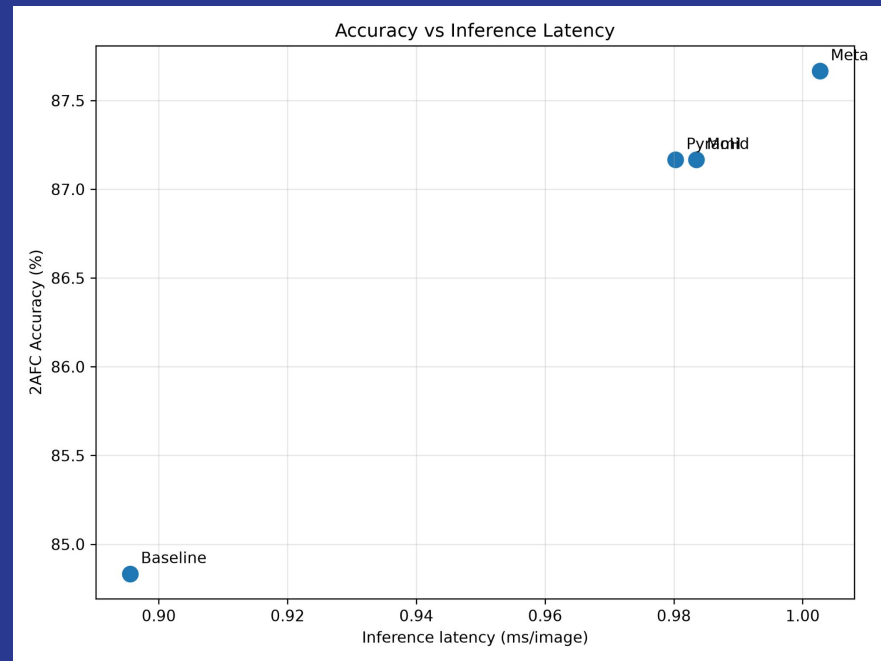
Pyramid-every increases from 0.90 to 0.98 (+8.889 %)

Meta-every increases from 0.90 to 1.00 (+11.111 %)



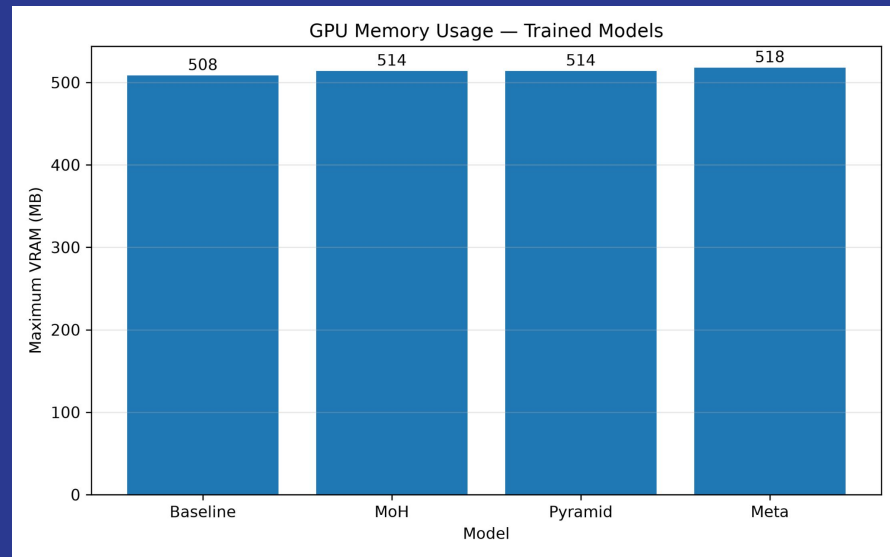
Trade-offs

All three efficient attention variants improved the accuracy considerably(around 3%) whereas increases the inference latency of only around 10%



Memory Usage

For the experiment setup with only 4k image triplets, peak GPU memory usage is almost the same for all four configurations, so, the effect can be ignored.



Conclusions

Vision Transformers equipped with efficient attention variants

- improved predictive performance than the baseline model.
- achieved best results from every-other-layer replacement
- Despite their theoretically reduced computational complexity, the efficient variants exhibited slightly higher inference latency.

Future Work

- Scalability: evaluate across different ViT sizes and input resolutions.
- Generalization: validate on larger datasets instead of only a subset.
- Inference optimization: reduce routing and implementation overhead.
- Adaptive layer selection: dynamically determine where efficient attention should be applied.



Thank you for listening